# Inventory Management System

## Why are Data Structures and Algorithms Essential in Handling Large Inventories?

Data Structures and Algorithms play a vital role in inventory management systems, especially when dealing with large numbers of products.

### 1. Fast Access and Updates

A warehouse may contain thousands of products. Efficient data structures allow products to be searched, added, updated, and removed quickly, improving overall system performance.

### 2. Efficient Memory Usage

Choosing the appropriate data structure helps manage memory effectively, reducing unnecessary storage overhead and improving application efficiency.

### 3. Scalability

As the inventory grows, the system should continue to perform efficiently. Well-designed algorithms ensure that operations remain fast even with large datasets.

---

## Suitable Data Structures for Inventory Management

### ArrayList

- Suitable for small inventories.
- Maintains insertion order.
- Searching, updating, and deleting products by `productId` requires traversing the list.

**Time Complexity**

- Search: O(n)
- Update: O(n)
- Delete: O(n)

### HashMap

- Ideal for large inventories.
- Uses `productId` as the key and `Product` as the value.
- Provides direct access to products.

**Time Complexity**

- Search: O(1)
- Update: O(1)

- Delete: O(1)

---

# Time Complexity Analysis (Using ArrayList)

## 1. Add Product

```
public void addProduct(Product product){
    products.add(product);
    System.out.println("Product Added Successfully");
}
```

**Time Complexity: O(1)**

Adding an element to the end of an `ArrayList` is generally a constant-time operation.

---

## 2. Update Product

```
public void updateProduct(Product product) {
    for(Product prod : products){
        if(prod.getProductId() == product.getProductId()){
            prod.setProductName(product.getProductName());
            prod.setQuantity(product.getQuantity());
            prod.setPrice(product.getPrice());
            return;
        }
    }

    System.out.println("No Product Found");
}
```

**Time Complexity: O(n)**

The system must search through the list to find the matching `productId`. In the worst case, every product may need to be checked.

---

## 3. Delete Product

```
public void removeProduct(int productId){
    for(Product product : products){
        if(product.getProductId() == productId){
            products.remove(product);
            return;
```

```
        }
    }

    System.out.println("No Product Found");
}
```

**Time Complexity: O(n)**

The product must first be located using a linear search. After removal, the remaining elements may need to be shifted, resulting in O(n) complexity.

---

# Optimization Using HashMap

To improve performance, a `HashMap<Integer, Product>` can be used where:

- Key = `productId`
- Value = `Product` object

This allows direct access to products without traversing the entire collection.

## 1. Add Product

```
public void addProduct(Product product) {
    productsMap.put(product.getProductId(), product);
}
```

**Time Complexity: O(1)**

Insertion into a `HashMap` is constant time on average.

---

## 2. Update Product

```
public void updateProduct(int productId,
                          String productName,
                          int quantity,
                          double price) {

    Product product = productsMap.get(productId);

    if(product != null){
        product.setProductName(productName);
        product.setQuantity(quantity);
        product.setPrice(price);

        System.out.println("Product with ID " +
```

```
                            productId +
                            " Updated Successfully");
    }
    else{
        System.out.println("Invalid Product ID");
    }
}
```

**Time Complexity: O(1)**

Retrieving a product using its key from a `HashMap` is constant time on average.

---

## 3. Delete Product

```
public boolean deleteProduct(int productId) {

    if(productsMap.remove(productId) != null){
        System.out.println("Product with ID " +
                            productId +
                            " Removed Successfully");
        return true;
    }

    System.out.println("Invalid Product ID");
    return false;
}
```

**Time Complexity: O(1)**

Removing a product from a `HashMap` is constant time on average.

---

# Conclusion

The current implementation using `ArrayList` is suitable for small inventories but becomes inefficient as the number of products increases because update and delete operations require O(n) time.

For large-scale inventory systems, `HashMap` is the preferred data structure because it provides O(1) average time complexity for add, update, search, and delete operations, resulting in better performance and scalability.